LastMile

Delivery Route Optimizer

CS3342 Software Design Project

Optimizing delivery routes using heuristic algorithms & design patterns

Group Members — Team 26

Project: LastMile — Delivery Route Optimizer

Lessov Akniyet

Project Lead & Architect

- Project planning & management
- System architecture design
- Strategy pattern (algorithm/)
- 2-Opt algorithm implementation
- Maven build & CI/CD pipeline

Bulatov Smadiyar

Backend Developer

- Nearest Neighbor algorithm
- Greedy Edge + Union-Find
- Route & DeliveryGraph models
- Capacity splitting logic
- Input validation & exceptions

Naumov Iusuf

QA & Observer Module

- Observer pattern (observer/)
- ConsoleLogger & FileLogger
- ConstraintChecker implementation
- JUnit 5 test suite (30+ tests)
- Test strategy & coverage

Tursynov Birzhan

I/O & Reporting

- Template Method (report/)
- Console & File ReportGenerator
- CsvMatrixReader & JsonReader
- User Manual & Install Guide

Introduction

What does it do?

A Java CLI application that computes optimized delivery routes.

Compares 3 routing algorithms on a given distance matrix.

Handles real-world constraints:
- Vehicle capacity limits
- Time window enforcement
- Delivery priority ordering

Who are the users?

Logistics companies planning daily delivery schedules.

Fleet managers optimizing driver routes.

Route planners seeking to minimize fuel costs and delivery time.

Why this project?

Manual route planning is inefficient and costly.

Optimized routing reduces:
- Fuel costs
- Delivery time
- CO2 emissions

Based on the TSP — a classic NP-hard optimization problem.

Project Objective

Build a modular, extensible route optimization CLI tool that compares multiple heuristic algorithms under real-world delivery constraints.

Load Inputs

CSV distance matrices
JSON package data

3 Algorithms

· Nearest Neighbor
· Greedy Edge
· 2-Opt

Capacity Splitting

Multi-trip routing when vehicle weight exceeded

Constraint Checking

Time windows & priority enforcement

Report Generation

Output with comparison tables

Event Logging

Real-time observer-based console & file logging

Use Case Modeling

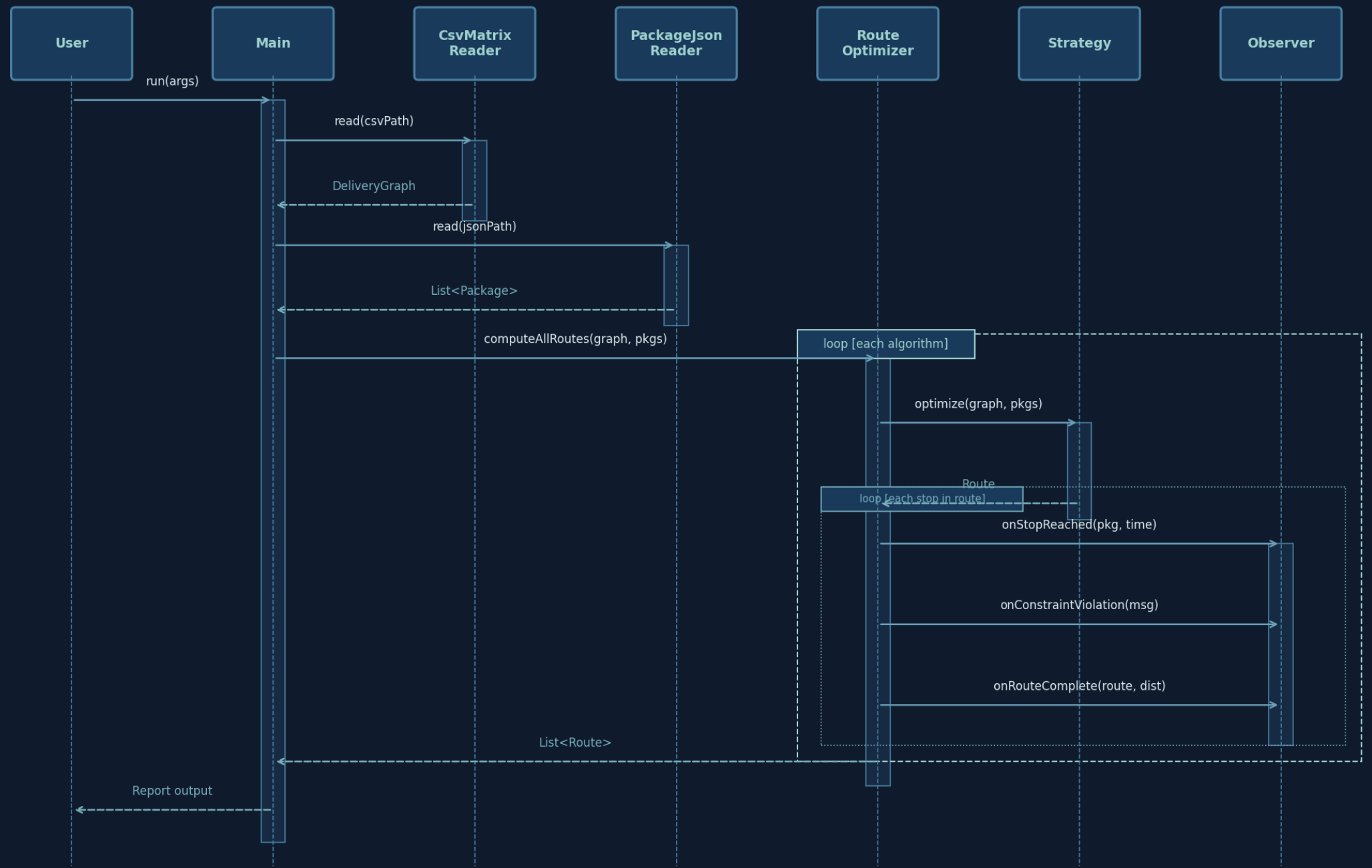
Primary Actor: Route Planner / Fleet Manager

UC1	**Load Distance Matrix**	Import CSV file with inter-stop distances
UC2	**Load Package Data**	Import JSON with delivery details, priorities, time windows
UC3	**Select Algorithm**	Choose NN, Greedy Edge, 2-Opt, or compare all three
UC4	**Set Capacity Constraint**	Define vehicle weight limit for multi-trip splitting
UC5	**Run Optimization**	Compute optimal delivery routes with selected algorithm
UC6	**View Report**	See route sequence, distances, time windows, violations
UC7	**Export Report**	Save detailed results to output file

Includes: Input validation · Constraint checking · Multi-trip splitting · Observer notifications

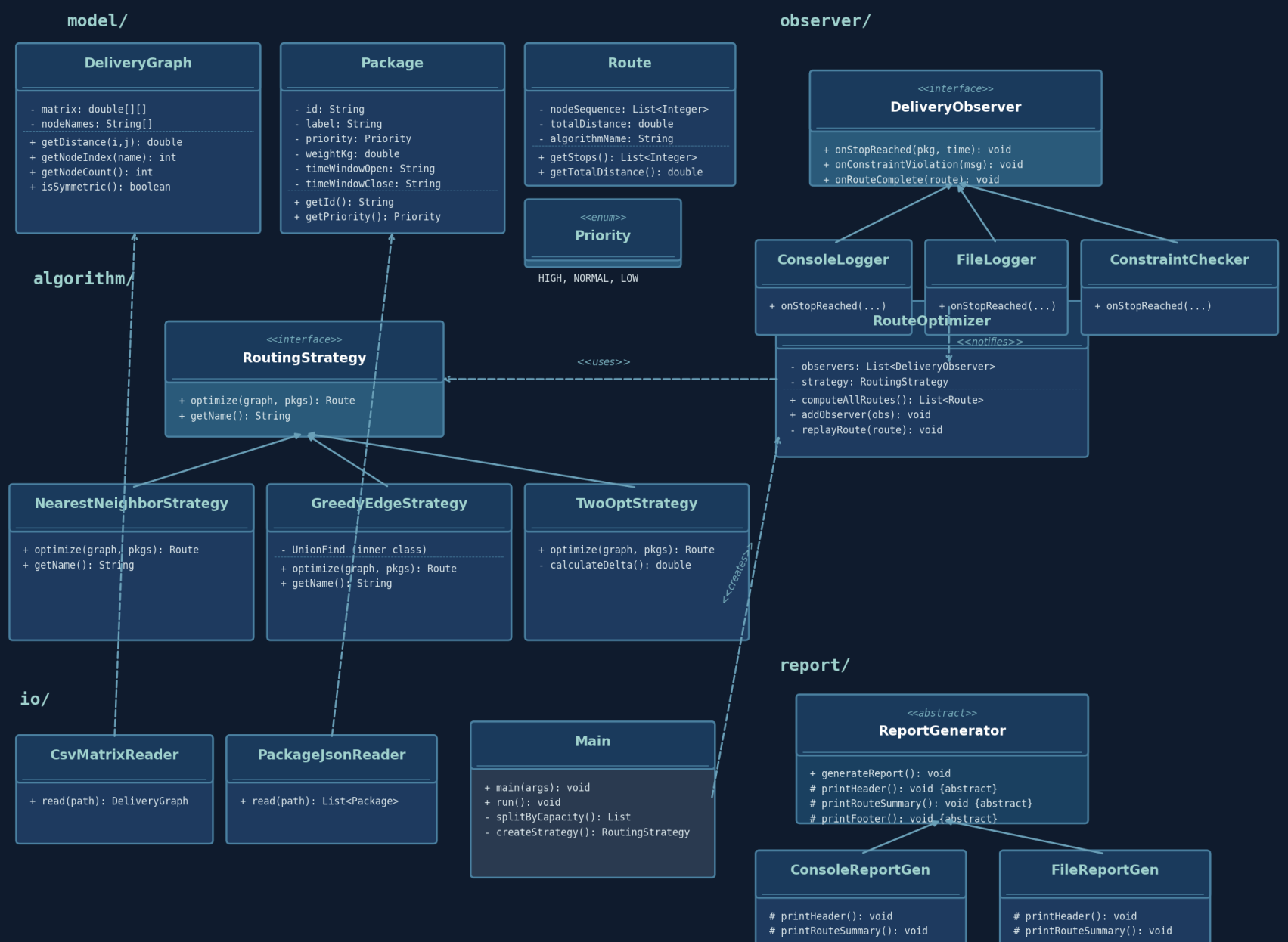
System Prototype & Demo

Sequence Diagram — Route Optimization Flow



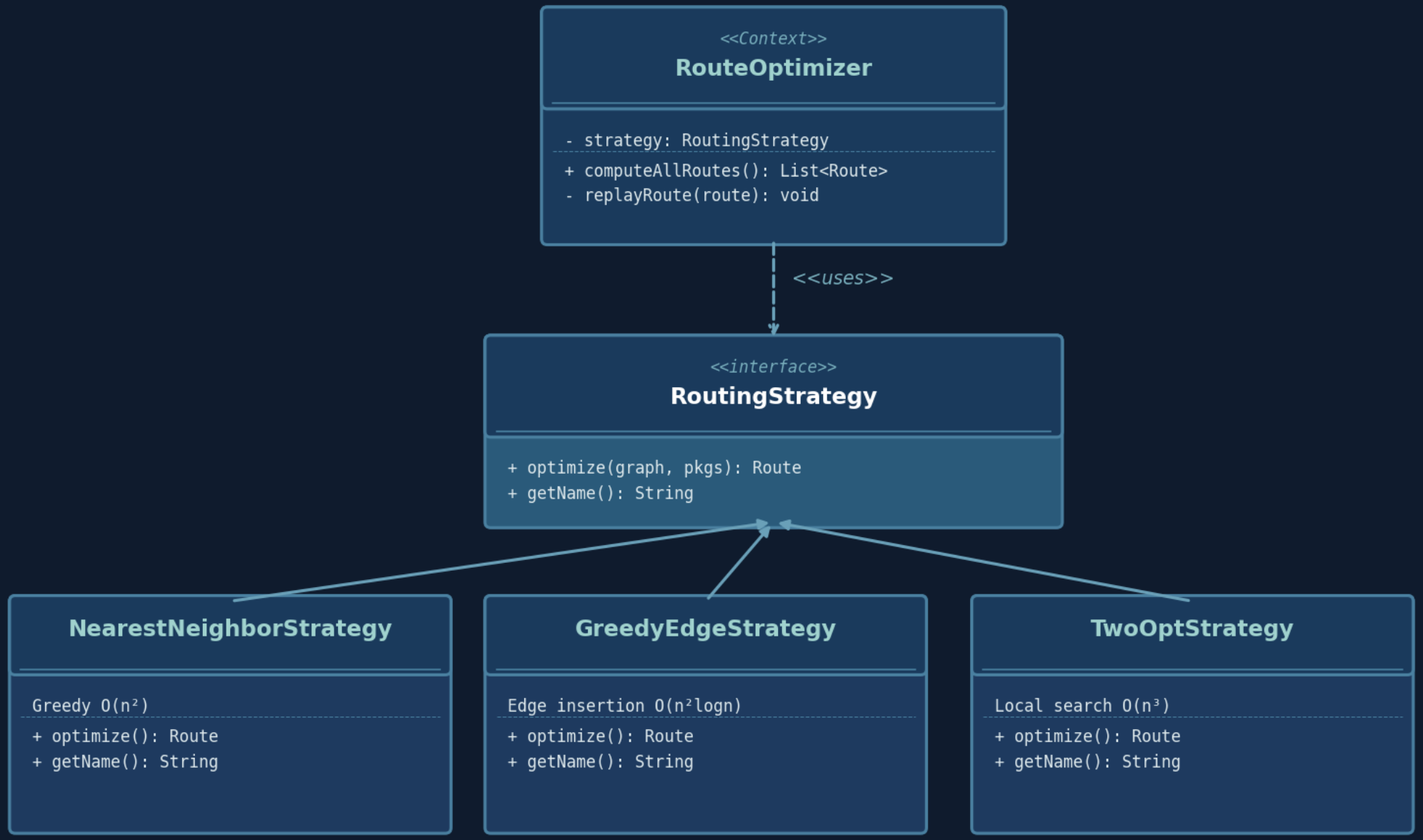
Class Diagram Design

Full UML class diagram showing all packages, interfaces, and relationships



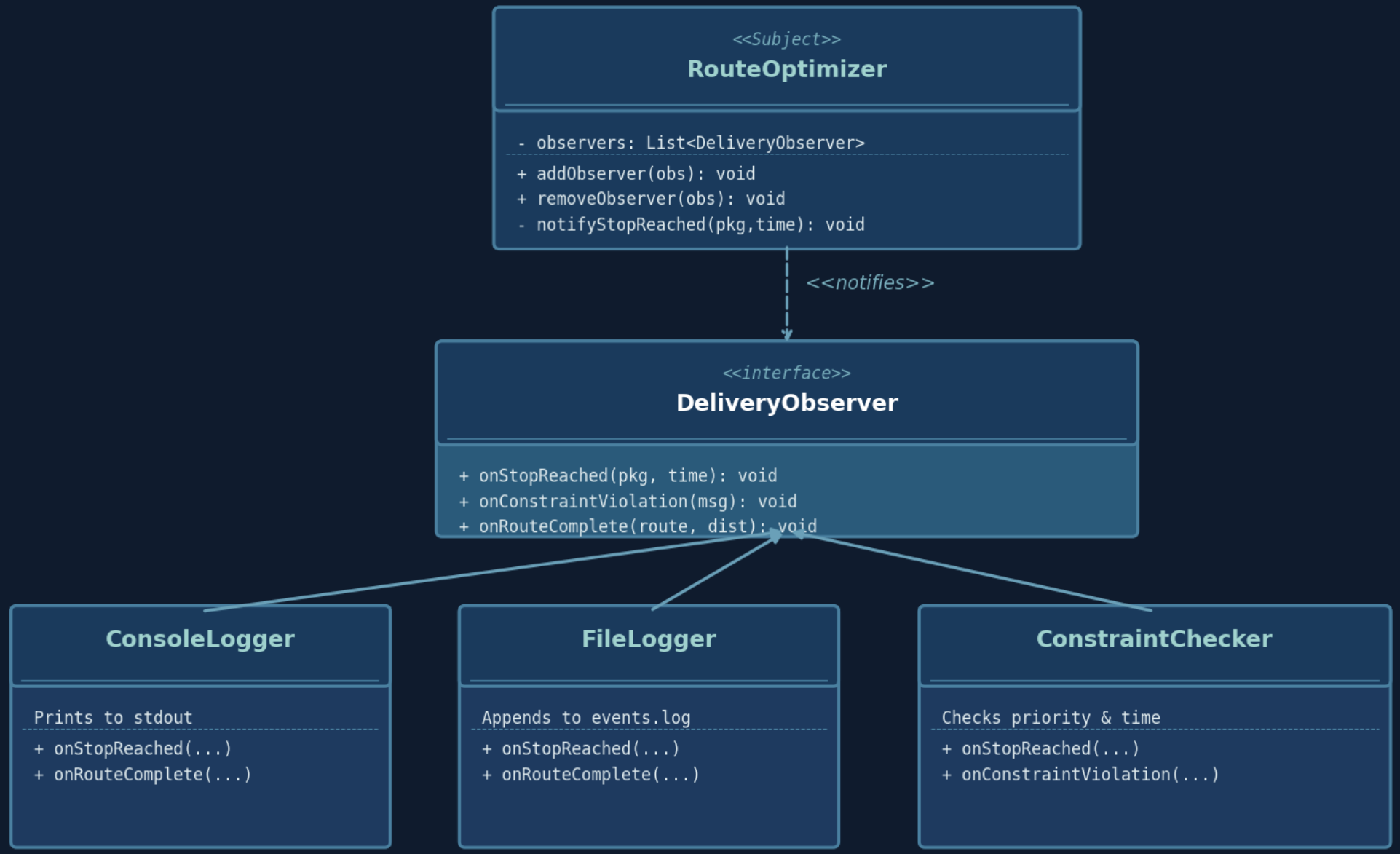
Strategy Pattern

Encapsulate interchangeable routing algorithms behind a common interface



Observer Pattern

Notify multiple listeners when delivery events occur during route replay



Template Method Pattern

Define report generation skeleton, let subclasses override formatting

<<abstract>>

ReportGenerator

```
+ generateReport() {template method}
    → printHeader()
    → printRouteSummary()
    → printStopBreakdown()
    → printViolations()
    → printComparison()
    → printFooter()
# printHeader() {abstract}
# printRouteSummary() {abstract}
# printFooter() {abstract}
```

ConsoleReportGenerator

```
Output: System.out (stdout)
# printHeader(): bordered title
# printRouteSummary(): formatted table
# printStopBreakdown(): tabular stops
# printViolations(): violation list
# printFooter(): closing border
```

FileReportGenerator

```
Output: file (Docs/last_report.txt)
# printHeader(): file title
# printRouteSummary(): file format
# printStopBreakdown(): file table
# printViolations(): file list
# printFooter(): timestamp + close
```

Design Principles Applied

1 Open-Closed Principle (OCP)

Open for extension, closed for modification

RoutingStrategy interface allows adding new algorithms (e.g. SimulatedAnnealing) without modifying RouteOptimizer.

```
RoutingStrategy
 ├ NearestNeighbor ✓
 ├ GreedyEdge ✓
 ├ TwoOpt ✓
 └ [NewAlgorithm] ✓  ← extend
```

2 Single Responsibility Principle (SRP)

Each class has one reason to change

CsvMatrixReader only reads CSV.
ConstraintChecker only validates constraints.
ReportGenerator only generates reports.

```
CsvMatrixReader → CSV parsing
ConstraintChecker → validation
ConsoleLogger → logging
ReportGenerator → reports
```

3 Liskov Substitution Principle (LSP)

Subtypes must be substitutable for base types

Any RoutingStrategy implementation can replace another without breaking RouteOptimizer.computeAllRoutes().

```
RouteOptimizer works with:
 NN ↔ Greedy ↔ TwoOpt
All return valid Route objects.
```

4 Dependency Inversion Principle (DIP)

Depend on abstractions, not concretions

RouteOptimizer depends on RoutingStrategy interface, not concrete algorithm classes. Same for DeliveryObserver.

```
RouteOptimizer
  → RoutingStrategy (interface)
  → DeliveryObserver (interface)
Not concrete classes.
```

Algorithm Details

Nearest Neighbor

$O(n^2)$

Greedy heuristic — always visit the closest
unvisited stop from current position.

Fast but often suboptimal (no backtracking).
Used as baseline and seed for 2-Opt.

Greedy Edge Insertion

$O(n^2 \log n)$

Sort all edges by distance, greedily add shortest valid edges.

Uses Union-Find for cycle detection.
Degree constraint ≤ 2 per node.

2-Opt Improvement

$O(n^3)$ iterative

Starts with NN solution, iteratively reverses sub-segments.

If reversing [i..j] reduces distance → swap.
Repeats until convergence (local optimum).

Performance Comparison (6-node graph)

· Nearest Neighbor: 95.25 km 143 min · Greedy Edge: 89.50 km 134 min · 2-Opt: 87.50 km 131 min

Test Case Strategy

JUnit 5 · 30+ Tests · 8 Test Classes · All Passing

Test Class	# Tests	What's Tested
NearestNeighborStrategyTest	5	Basic routing, equal distances, single stop, depot return
GreedyEdgeStrategyTest	8	All stops visited, correct distance, depot first/last
TwoOptStrategyTest	5	Improves on NN, already optimal, single stop valid
DeliveryGraphTest	7	Symmetric/asymmetric, non-square, negative distances
CsvMatrixReaderTest	6	Valid parse, non-square throws, missing file
ConstraintCheckerTest	7	Priority ordering, time windows, early/late arrival

Team Work

Challenges Faced

1. Greedy Edge with Union-Find
   Implementing cycle detection with disjoint sets and degree constraints was complex.

2. Multiple Constraint Types
   Balancing capacity, time windows, and priority ordering simultaneously.

3. Extensibility
   Designing for easy addition of new algorithms without breaking existing code.

4. Coordination & Integration
   Merging work from 4 members across algorithm, observer, and reporting modules.

How We Addressed Them

Strategy Pattern
Plug-and-play algorithm architecture.
Each member implemented their strategy.

Observer Pattern
Decoupled logging from business logic.
Observer module developed independently.

Template Method Pattern
Standardized report sections.
I/O team only overrides formatting.

Regular Team Meetings
Weekly sync to review progress, resolve integration issues, and plan next steps.

Lifecycle Management

Build System: Maven

maven-compiler-plugin → Java 11 compilation

maven-surefire-plugin → automated test execution

maven-shade-plugin → fat JAR to Release/

Dependencies:

Gson 2.10.1 (JSON) · JUnit 5.10.2 (Testing)

CI/CD Pipeline

Build: ./mvnw clean package

Test: ./mvnw test (30+ tests, all passing)

Package: JAR → Release/DeliveryRouteOptimizer.jar

Version Control: Git + GitHub

Structured commits · Feature branching

Project Structure

Source/	Maven project (src/main/java + src/test/java)	8 packages, 15+ classes
Release/	Compiled JAR (DeliveryRouteOptimizer.jar)	Ready-to-run artifact
data/	Sample input files (distances.csv, packages.json)	Test data included
Docs/	User Manual, Installation Guide, Reports	Full documentation suite
logs/	Runtime event logs (events.log)	Observer-generated

Conclusion

- ✓ **Modular Architecture** — Clean separation via Strategy, Observer, Template Method patterns
- ✓ **3 Algorithms** — Nearest Neighbor, Greedy Edge Insertion, 2-Opt Improvement
- ✓ **Real-world Constraints** — Vehicle capacity, time windows, delivery priorities
- ✓ **Comprehensive Testing** — 30+ JUnit 5 tests across 8 test classes
- ✓ **Production-ready CLI** — Fat JAR with Maven build, sample data, demo script

Acknowledgement of AI usage: AI tools were used to brainstorm project ideas and assist with code generation during the problems faced